

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 1: Desarrollo de aplicaciones web en el entorno servidor

Lección 1: El proceso del desarrollo de software

Análisis y especificación de requisitos

Diseño

Implementación. Conceptos generales de desarrollo de software

Validación y verificación de sistemas

Análisis y especificación de requisitos

El análisis de requisitos es una fase clave en el desarrollo de software donde se definen y documentan las necesidades del sistema. Su objetivo es garantizar que el software desarrollado cumpla con las expectativas del cliente y los usuarios finales.

1. Tipos de requisitos

Los requisitos del software se pueden clasificar en:

- **Requisitos funcionales:** Describen las funciones y características específicas que el software debe proporcionar.
Ejemplo: Un sistema de reservas de hotel debe permitir a los usuarios buscar disponibilidad y realizar reservas.
- **Requisitos no funcionales:** Definen restricciones o criterios de calidad que el sistema debe cumplir.
Ejemplo: La aplicación web debe responder en menos de 2 segundos.
- **Requisitos técnicos:** Especifican tecnologías y estándares que se deben utilizar.
Ejemplo: El servidor debe ejecutar PHP 8 y la base de datos debe ser MySQL.
- **Requisitos de usuario:** Reflejan las expectativas y necesidades del usuario final.
Ejemplo: La interfaz debe ser accesible y usable en dispositivos móviles.

2. Modelos para el análisis de requisitos

Algunos modelos utilizados para el análisis de requisitos incluyen:

- **Casos de uso:** Representan las interacciones entre los usuarios y el sistema.
- **Historias de usuario:** Definen requisitos desde la perspectiva del usuario final en formato “Como [usuario], quiero [acción] para [beneficio]”.
- **Diagramas de flujo de datos:** Representan cómo se mueven los datos dentro del sistema.

Ejemplo práctico:

En un sistema de tienda online, una *historia de usuario* podría ser:

“Como cliente, quiero poder añadir productos al carrito para comprarlos más tarde.”

3. Documentación de requisitos

Se utilizan documentos como:

- SRS (Software Requirement Specification): Documento formal con los requisitos detallados.
- Historias de usuario en metodologías ágiles.

Ejemplo: Un SRS incluiría una sección con el siguiente requisito:

“El sistema debe permitir a los usuarios registrarse con un correo y una contraseña segura.”

4. Validación de requisitos

Se debe verificar que los requisitos sean completos, coherentes y alcanzables.

Métodos:

- Revisión con el cliente.
- Creación de prototipos.
- Pruebas tempranas en el desarrollo.

5 Gestión de requisitos

Consiste en realizar un seguimiento y priorización de los requisitos durante el proyecto. Se pueden usar herramientas como Jira o Trello.

Ejemplo: Se define que las funcionalidades esenciales (como la autenticación) tendrán prioridad alta, mientras que el soporte para redes sociales será una prioridad media.

Diseño

El diseño del software define la estructura y arquitectura del sistema antes de su implementación.

1. Modelos para el diseño de sistemas

Los modelos más utilizados incluyen:

- **Arquitectura en capas:** Divide la aplicación en presentación, lógica de negocio y datos.
- **Modelo MVC (Modelo-Vista-Controlador):** Separa la lógica de negocio, la interfaz de usuario y el control del flujo de datos.

Ejemplo práctico: Un sistema de gestión de empleados puede utilizar MVC:

- **Modelo:** Empleado.php (gestiona datos del empleado).
- **Vista:** empleados.php (muestra los empleados).
- **Controlador:** EmpleadoController.php (maneja la lógica entre el modelo y la vista).

2. Diagramas de diseño. El estándar UML

UML es un lenguaje estándar para representar visualmente sistemas de software. Diagramas importantes:

- **Diagramas de clases:** Representan las relaciones entre las clases.
- **Diagramas de secuencia:** Muestran el flujo de ejecución de una funcionalidad.

Ejemplo:

Para un sistema de usuarios, un diagrama de clases mostraría las relaciones entre las clases Usuario, Administrador y Cliente.

3. Documentación

La documentación debe incluir descripciones de los componentes, arquitectura y API del sistema. Se pueden usar herramientas como Swagger o PHPDoc para generar documentación automática del código.

La documentación es una parte fundamental del desarrollo de software, ya que proporciona información clara sobre la estructura, componentes y funcionamiento del sistema. Esto facilita la colaboración en equipo, el mantenimiento del código y la escalabilidad del proyecto.

Tipos de Documentación

1. **Documentación Técnica:** Explica la arquitectura, las tecnologías utilizadas y las APIs del sistema.
2. **Documentación del Código:** Comentarios dentro del código fuente que facilitan su comprensión y mantenimiento.
3. **Documentación del Usuario:** Guías o manuales para ayudar a los usuarios a interactuar con el sistema.

Implementación. Conceptos generales de desarrollo de software

1. Principios básicos del desarrollo de software

Los principios clave incluyen:

- **Modularidad:** Dividir el código en partes reutilizables.
- **Reutilización:** Aprovechar código ya existente.
- **Simplicidad:** Escribir código limpio y fácil de entender.

Ejemplo práctico en PHP:

Crear una clase Usuario para gestionar usuarios en lugar de repetir código en distintos archivos.

2. Técnicas de desarrollo de software

Se pueden aplicar:

- **Programación estructurada:** Uso de funciones y estructuras básicas.
- **Programación orientada a objetos (POO):** Uso de clases y objetos.

Ejemplo práctico:

- Código en PHP estructurado:

```
function calcularEdad($añoNacimiento) {  
    return date("Y") - $añoNacimiento;  
}  
echo calcularEdad(1995);
```

- Código en PHP orientado a objetos:

```
class Persona {  
    public $nombre;  
    public $añoNacimiento;  
  
    public function __construct($nombre, $añoNacimiento) {  
        $this->nombre = $nombre;  
        $this->añoNacimiento = $añoNacimiento;  
    }  
  
    public function calcularEdad() {  
        return date("Y") - $this->añoNacimiento;  
    }  
}  
$persona = new Persona("Juan", 1995);  
echo $persona->calcularEdad();
```

Validación y verificación de sistemas

1. Planificación

Se define cómo se van a probar las distintas partes del software. Herramientas comunes:

- Casos de prueba.
- Pruebas unitarias.

Ejemplo: En un sistema de login, se definen pruebas como:

✓ El usuario ingresa credenciales correctas y accede al sistema.

✗ El usuario ingresa una contraseña incorrecta y recibe un mensaje de error.

2. Métodos formales de verificación

Se utilizan técnicas como:

- Revisiones de código.
- Pruebas de caja blanca (analizar la lógica interna).

3. Métodos automatizados de análisis

Se emplean herramientas que automatizan la validación:

- **PHPUnit:** Para realizar pruebas unitarias en PHP.
- **SonarQube:** Para analizar calidad de código.

Ejemplo en PHP: Prueba unitaria con PHPUnit:

```
use PHPUnit\Framework\TestCase;

class CalculadoraTest extends TestCase {
    public function testSuma() {
        $this->assertEquals(5, 2 + 3);
    }
}
```

5. IA en pruebas de software

Uso de IA para detectar errores en el código.

- Herramientas como GitHub Copilot o ChatGPT para sugerencias de código.
- Pruebas automatizadas con IA.

IntelliSense y herramientas de asistencia en desarrollo

- Uso de IntelliSense en Visual Studio Code para mejorar la productividad.
- Plugins como PHP Intelephense y PHP Debugger.
- Detección temprana de errores con linters y autocompletado inteligente.